

UNIT – III

1. Explain Process State Transition with Neat Diagram

Definition:

A process is a program in execution. During execution, a process moves through different states.

Process States:

New

Ready

Running

Waiting (Blocked)

Terminated

Diagram: Reffer class notes

Explanation:

New → Ready: Process admitted into memory.

Ready → Running: CPU assigned.

Running → Waiting: Waiting for I/O.

Waiting → Ready: I/O completed.

Running → Terminated: Process finishes execution.

2. What is PCB? Explain with Diagram

Definition:

PCB (Process Control Block) is a data structure maintained by the operating system to store information about a process.

Diagram: Reffer class notes

Components:

Process ID

Process State

Program Counter

CpU Registers

Memory Information

I/O Information

Scheduling Information

Importance:

Supports context switching.

Maintains process information.

3. Explain Process Scheduling

Definition

Process Scheduling is the activity of selecting a process from the ready queue and assigning CPU to it.

Objectives:

Maximize CPU utilization.

Increase throughput.

Reduce waiting time.

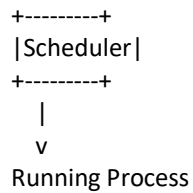
Improve response time.

Diagram:

Ready Queue

|

v



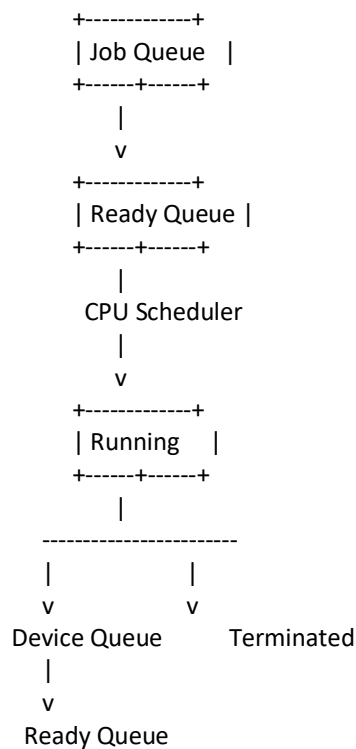
Types:

Long-Term Scheduling

Medium-Term Scheduling

Short-Term Scheduling

4. Explain Queuing Diagram Representation of Process Scheduling Diagram



Explanation:

Job Queue contains all processes.

Ready Queue contains processes waiting for CPU.

Device Queue contains processes waiting for I/O.

5. Write a Note on Schedulers

Definition:

Schedulers select processes for execution.

Types:

Long-Term Scheduler::

Selects processes from job pool.

Controls degree of multiprogramming.

Short-Term Scheduler::

Selects process from ready queue.

Allocates CPU frequently.

Medium-Term Scheduler::

Swaps processes in and out of memory.

Comparison

Scheduler	Speed	Function
Long-Term	Slow	Admit jobs
Medium-Term	Moderate	Swapping
Short-Term	Fast	CPU Allocation

6. Write a Note on Cooperating Processes

Definition:

Processes that can affect or be affected by other processes are called cooperating processes.

Reasons for Cooperation:

Information Sharing

Computation Speedup

Modularity

Convenience

Example:

Process A ----> Shared Data <---- Process B

Advantages:

Faster execution.

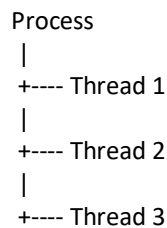
Resource sharing.

7. Write a Note on Threads

Definition:

A thread is the smallest unit of CPU execution within a process.

Diagram



Types:

- User-Level Threads

Managed by user library.

- Kernel-Level Threads

Managed by operating system.

Advantages:

Faster execution.

Better responsiveness.

8. Explain the Benefits of Threads

1. Responsiveness

Application remains responsive.

2. Resource Sharing

Threads share memory and resources.

3. Economy

Less overhead than processes.

4. Scalability

Can utilize multiple processors.

5. Faster Context Switching

Switching threads is faster than processes.

(Explain in detail)

9. Explain Preemptive and Non-Preemptive Scheduling

Preemptive Scheduling:

Definition

CPU can be taken away from a running process.

Examples

Round Robin

SRTF

Priority Scheduling

Advantages

Better response time.

Suitable for time-sharing systems.

Disadvantages

Higher overhead.

Non-Preemptive Scheduling:

Definition:

Once CPU is assigned, process keeps CPU until completion.

Examples:

FCFS

Non-Preemptive SJF

Advantages:

Simple implementation.

Less overhead.

Disadvantages

Poor response time.

10. List and Explain Various Scheduling Criteria

1. CPU Utilization

Percentage of time CPU remains busy.

$\text{CPU Utilization} = \frac{\text{Busy Time}}{\text{Total Time}}$

Goal: Maximize.

2. Throughput

Number of processes completed per unit time.

Goal: Maximize.

3. Turnaround Time

Time between process submission and completion.

$\text{Turnaround Time} = \text{Completion} - \text{Arrival}$

Goal: Minimize.

4. Waiting Time

Time spent in ready queue.

$\text{Waiting Time} = \text{Turnaround} - \text{Burst}$

Goal: Minimize.

5. Response Time

Time between submission and first response.

$\text{Response Time} = \text{First Response} - \text{Arrival}$

Goal: Minimize.

6. Fairness

Each process should receive a fair share of CPU time.

7. Predictability

System should behave consistently.

11. Explain FCFS Scheduling with an Example

Definition

First Come First Serve (FCFS) scheduling executes processes in the order of their arrival.

Characteristics:

Non-preemptive scheduling.

Simple and easy to implement.

Uses a FIFO queue.

Example

Process Burst Time

P1 24

P2 3

P3 3

Gantt Chart

0 24 27 30

| P1 | P2 | P3 |

Waiting Time

Process Waiting Time

P1 0

P2 24

P3 27

Average Waiting Time

$\{0+24+27\}/\{3\}=17$

Advantages:

Simple.

Fair allocation.

Disadvantages:

Convoy effect.

Poor average waiting time.

12. Explain Shortest Job First (SJF) Scheduling with Example

Definition

SJF selects the process with the smallest CPU burst time first.

Example

Process Burst Time

P1 6

P2 8

P3 7

P4 3

Order

P4 → P1 → P3 → P2

Gantt Chart

0 3 9 16 24

| P4 | P1 | P3 | P2 |

Waiting Time

Process Waiting Time

P4 0

Process Waiting Time

P1 3

P3 9

P2 16

Average Waiting Time

$\{0+3+9+16\}/4=7$

Advantages:

Minimum average waiting time.

Disadvantages:

Difficult to predict burst time.

Possible starvation.

13. Explain Round Robin Scheduling with Example

Definition:

Round Robin assigns CPU to processes for a fixed time quantum.

Example:

Time Quantum = 2

Process Burst Time

P1 5

P2 4

P3 2

Gantt Chart

0 2 4 6 8 10 11

|P1|P2|P3|P1|P2|P1|

Advantages:

Good response time.

Suitable for time-sharing systems.

Disadvantages

Context switching overhead.

14. Explain Priority Scheduling with Example

Definition:

CPU is allocated to the process with highest priority.

Example

Process Burst Priority

P1 10 3

P2 1 1

P3 2 4

P4 1 5

(Higher number = Higher priority)

Execution Order

P4 → P3 → P1 → P2

Gantt Chart

0 1 3 13 14

|P4|P3| P1 |P2|

Advantages:

Important jobs executed first.

Disadvantages:

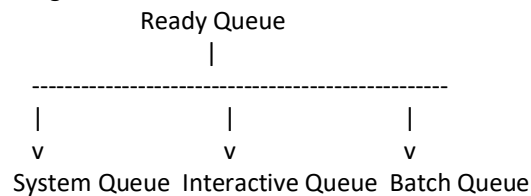
Starvation of low-priority processes.

15. Explain Multilevel Queue Scheduling

Definition

Ready queue is divided into several queues based on process type.

Diagram



Scheduling:

Each queue has its own algorithm.

Fixed priority among queues.

Advantages:

Better organization.

Disadvantages:

Starvation possible.

16. Explain Multilevel Feedback Queue Scheduling

Definition

Processes can move between queues based on behavior.

Diagram:

Queue 1 (RR, 8ms)

```
graph TD;
    A[Queue 1 (RR, 8ms)] --> B[Queue 2 (RR, 16ms)];
    B --> C[Queue 3 (FCFS)];
```

The diagram shows a vertical flow of three queues. The top queue is 'Queue 1 (RR, 8ms)'. A vertical line with an arrowhead points down from it to the middle queue, 'Queue 2 (RR, 16ms)'. Another vertical line with an arrowhead points down from 'Queue 2' to the bottom queue, 'Queue 3 (FCFS)'.

Features:

New processes enter highest queue.

CPU-intensive processes move downward.

Interactive processes stay at higher levels.

Advantages:

Flexible.

Prevents starvation.

Disadvantages:

Complex implementation.

17. Difference Between Long-Term, Medium-Term and Short-Term Schedulers

Feature	Long-Term	Medium-Term	Short-Term
Function	Admit jobs	Swapping	CPU allocation
Speed	Slow	Moderate	Fast
Frequency	Less frequent	Occasional	Very frequent

Feature	Long-Term	Medium-Term	Short-Term
Controls	Multiprogramming	Memory usage	CPU usage

18. Explain Multiprocessor Scheduling

Definition:

Scheduling in systems containing multiple CPUs.

Types:

Asymmetric Multiprocessing

One master CPU schedules.

Other CPUs execute tasks.

Symmetric Multiprocessing (SMP)

All processors schedule themselves.

Advantages:

Increased throughput.

Better reliability.

Parallel execution.

Challenges:

Load balancing.

Processor affinity.

19. Round Robin Numerical

Given

Process Burst Time

P1 6

P2 8

P3 7

P4 3

Time Quantum = 5 ms

Gantt Chart

0 5 10 15 18 19 22 25

| P1 | P2 | P3 | P4 | P1 | P2 | P3 |

Completion Times

Process Completion

P4 18

P1 19

P2 22

P3 25

Waiting Time

WT = Completion - Burst

Process WT

P1 19-6=13

P2 22-8=14

P3 25-7=18

P4 18-3=15

Average Waiting Time

$\{13+14+18+15\}/\{4\}=15\text{ms}$

20. SJF Numerical

Given

Process Burst Time

Process Burst Time

P1 15
P2 4
P3 10
P4 8
P5 5

Execution Order (SJF)

P2 → P5 → P4 → P3 → P1

Gantt Chart

0 4 9 17 27 42
|P2|P5|P4| P3 | P1 |

Turnaround Time

Process TAT

P2 4
P5 9
P4 17
P3 27
P1 42

Average Turnaround Time

$\{4+9+17+27+42\}/\{5\}=19.8\text{ms}$

21. Scheduling Algorithms Numerical

Given

Process Burst Priority

P1 2 2
P2 1 1
P3 8 4
P4 4 2
P5 5 3

(All arrive at time 0)

(a) FCFS

Gantt Chart

0 2 3 11 15 20
|P1|P2| P3 |P4|P5|

Turnaround Times

Process TAT

P1 2
P2 3
P3 11
P4 15
P5 20

(b) SJF

Order:

P2 → P1 → P4 → P5 → P3

Gantt Chart

0 1 3 7 12 20
|P2|P1|P4|P5|P3|

Turnaround Times

Process TAT

P2	1
P1	3
P4	7
P5	12
P3	20

(c) Non-Preemptive Priority

(Higher number = Higher Priority)

Order:

P3 → P5 → P1 → P4 → P2

Gantt Chart

0 8 13 15 19 20
| P3 | P5 | P1 | P4 | P2 |

Turnaround Times

Process TAT

P3	8
P5	13
P1	15
P4	19
P2	20

(d) Round Robin (Quantum = 2)

Gantt Chart

0 2 3 5 7 9 11 13 15 16 18 19 20
P1 P2 P3 P4 P5 P3 P4 P5 P3 P5 P3 P5

Turnaround Times

Process TAT

P1	2
P2	3
P4	13
P3	19
P5	20

UNIT – IV

1. What is Critical Section? What are the Requirements for a Solution to the Critical Section Problem?

Definition:

A Critical Section is a part of a program where shared resources (variables, files, memory, etc.) are accessed. Only one process should execute its critical section at a time.

Structure of a Process:

Do {

 Entry Section

 Critical Section

 Exit Section

 Remainder Section

} while (TRUE);

Requirements for a Correct Solution

1. Mutual Exclusion

Only one process can be in the critical section at a time.

2. Progress

If no process is in the critical section, the selection of the next process should not be postponed indefinitely.

3. Bounded Waiting

A process should not wait forever to enter the critical section.

Advantages:

Prevents race conditions.

Ensures data consistency.

2. Explain the Concept of Semaphores in Process Synchronization

Definition:

A Semaphore is a synchronization tool used to control access to shared resources.

Types of Semaphores

Binary Semaphore

Semaphore = 0 or 1

Used for mutual exclusion.

Counting Semaphore

Semaphore = 0,1,2,...

Used when multiple instances of resources exist.

Operations:

wait(P)

wait(S)

```
{  
    while(S <= 0);  
    S--;
```

```
}
```

signal(V)

signal(S)

```
{  
    S++;  
}
```

Example:

wait(mutex);

Critical Section

signal(mutex);

Advantages:

Prevents race conditions.

Supports process synchronization.

3. Explain Bounded Buffer Problem

Definition:

The Bounded Buffer Problem is a synchronization problem involving a producer and a consumer sharing a fixed-size buffer.

Diagram:

Producer ----> Buffer ----> Consumer

Semaphores Used:

mutex = 1

full = 0

empty = n

Producer

wait(empty);

wait(mutex);

Insert Item:

```
signal(mutex);
signal(full);
Consumer
wait(full);
wait(mutex);
```

Remove Item:

```
signal(mutex);
signal(empty);
```

Advantages:

Prevents buffer overflow.

Prevents buffer underflow.

4. What is Readers-Writers Problem? Explain

Definition

A database is shared among multiple readers and writers.

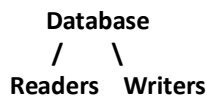
Rules:

Multiple readers can read simultaneously.

Only one writer can write at a time.

Readers and writers cannot access simultaneously.

Diagram



Reader Process:

```
wait(mutex);
readcount++;
if(readcount==1)
    wait(wrt);
signal(mutex);
```

Reading:

```
wait(mutex);
readcount--;
if(readcount==0)
    signal(wrt);
signal(mutex);
```

Advantages

Efficient resource utilization.

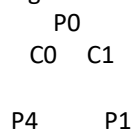
Allows concurrent reading.

5. Explain Dining Philosophers Problem

Definition:

Five philosophers sit around a table sharing five chopsticks.

Diagram:



```

    C4  C2
      P3
      |
      P2
Where:
P = Philosopher
C = Chopstick

```

Problem:
Each philosopher needs two chopsticks to eat.
Deadlock Situation
All philosophers pick one chopstick and wait for the second.

Solution:
Limit philosophers.
Use semaphores.
Number chopsticks

Applications:
Resource allocation problems.

6. Explain Any One Classic Problem of Synchronization

Producer-Consumer Problem:

Definition→Producer generates data and consumer consumes data.

Diagram:

```

    Producer
      |
      v
    Buffer
      |
      v
    Consumer

```

Issues:
Producer should not insert into a full buffer.
Consumer should not remove from an empty buffer.

Solution
Use semaphores:
mutex
full
empty

Advantages:
Proper synchronization.
No data loss.

7. What are the Necessary Conditions for Deadlock?

Deadlock occurs when a set of processes waits indefinitely for resources.

Four Necessary Conditions

1. Mutual Exclusion

At least one resource is non-shareable.

2. Hold and Wait

A process holds resources while waiting for others.

3. No Preemption

Resources cannot be forcibly taken away.

4. Circular Wait

$P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$

Conclusion

All four conditions must exist simultaneously for deadlock.

8. Explain Deadlock Prevention

Definition:

Deadlock prevention ensures that at least one necessary condition never occurs.

Methods:

- Prevent Mutual Exclusion

Use sharable resources where possible.

- Prevent Hold and Wait

Process requests all resources at once.

- Prevent No Preemption

Resources can be forcibly released.

- Prevent Circular Wait

Assign resource ordering.

Advantages:

Deadlock never occurs.

Disadvantages:

Low resource utilization.

9. Explain Resource Allocation Graph with Example

Definition:

A Resource Allocation Graph (RAG) is a directed graph representing processes and resources.

Symbols

Circle = Process

Square = Resource

Example

$P1 \text{ ----} \rightarrow R1$

$\begin{array}{c} \wedge \quad | \\ | \quad \vee \end{array}$

$R2 \text{ <----} P2$

Interpretation:

Process $\rightarrow$ Resource = Request edge

Resource $\rightarrow$ Process = Allocation edge

Deadlock Detection:

If a cycle exists:

$P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1$

Deadlock may occur.

10. Explain Deadlock Avoidance Using Safe State Algorithm

Safe State:

A system is in a safe state if there exists a sequence of processes that can complete successfully.

Example

Available Resources = 3

Process Need

P1 2

P2 1

P3 3

Available = 3

Safe Sequence:

$P2 \rightarrow P1 \rightarrow P3$

Algorithm:

- Find process whose need $\leq$ available.
- Execute it.
- Release allocated resources.
- Continue until all processes finish.

Advantages:

Avoids deadlock.

Maintains safe state.

11. Explain Deadlock Avoidance Using Resource Allocation Graph Algorithm

Applicable:

Single instance of each resource type.

Method:

Add Claim Edge before actual request.

Grant request only if no cycle is formed.

Example:

Claim Edge

$P1 \text{ -----} \rightarrow R1$

After request:

$P1 \text{ -----} \rightarrow R1$

Rule:

Grant resource only if graph remains acyclic.

Advantage:

Prevents unsafe states.

12. Explain Banker's Algorithm

Definition:

Banker's Algorithm is a deadlock avoidance algorithm proposed by Edsger W. Dijkstra.

Data Structures:

- Available: Available resources.
- Max: Maximum resource demand.
- Allocation: Currently allocated resources.
- Need
Need = Max - Allocation

Steps:

Check if request $\leq$ Need.

Check if request $\leq$ Available.

Temporarily allocate resources.

Run Safety Algorithm.

Grant request only if system remains safe.

Advantages:

Prevents deadlock.

Ensures safe execution.

Disadvantages:

Requires advance knowledge of maximum needs.

13. Explain How to Recover from Deadlock

Methods

1. Process Termination

Abort All Processes

P1

P2

P3

Terminate all.

Abort One Process at a Time

Select victim process and terminate.

2. Resource Preemption

Steps

Select victim process.

Preempt resources.

Rollback process.

Restart later.

Advantages:

Recovers system quickly.

Disadvantages:

Data loss possible.

14. Explain Deadlock Detection Algorithm for Single Instance of Each Resource Type

Resource Allocation Graph Method

Steps:

Construct Resource Allocation Graph.

Search for cycles.

Example

P1 → R1 → P2

↑ ↓

R2 ←----- P3

Rule:

No cycle → No deadlock.

Cycle exists → Deadlock exists.

Algorithm:

Build graph.

Apply cycle detection.

Report deadlocked processes.

Advantages:

Simple.

Easy to implement for single-instance resources.

Disadvantages:

Not suitable for multiple instances of resources.